

JAVASCRIPT PROGRAMMING — DIGITAL NOTES

Complete Reference Guide • ES6+ Features, DOM, Async/Await, Closures, OOP & Event Loop (With Code & Outputs)

1. Variables, Scope & Data Types

var vs let vs const

Keyword	Scope	Reassign	Hoisted
<code>var</code>	Function	Yes	Yes (undefined)
<code>let</code>	Block	Yes	TDZ (Error)
<code>const</code>	Block	No	TDZ (Error)



Temporal Dead Zone (TDZ)

Accessing `let` / `const` before initialization throws a `ReferenceError`.

Template Literals & I

```
// Array & Object D
const user = { name
const { name, role

const colors = ['re
const [first, ...ot

console.log(`${name
console.log(others)
```

OUTPUT:

```
Alex is a Dev
['green', 'blue']
```

Data Types & Equality

Primitives (Value) vs Reference Types (Memory Address).

```
// Primitive Types: Number, String, Boolean, null, undefined, Symbol, BigInt
console.log(typeof 42);           // "number"
console.log(typeof null);         // "object" (JS bug!)

// Loose (==) vs Strict (===) Equality
console.log('5' == 5);
console.log('5' === 5);
```

OUTPUT:

```
true
false
```

2. Functions, Arrow Functions & Closures

Arrow Functions vs Regular

```
// Arrow functions do NOT have their own 'this' or 'arguments'
const obj = {
  val: 10,
  regularFunc: function() {
    console.log(this.val);
  },
  arrowFunc: () => {
    console.log(this.val);
  }
}
```

Closures

A function bundled with references to its surrounding state (lexical environment).

```
function createCounter() {
  let count = 0;
  return function() {
    count++;
    return count;
  };
}
```

```
};

obj.regularFunc();
obj.arrowFunc();
```

OUTPUT:

```
10
undefined
```

```
};

}

const counter = createCounter();
console.log(counter());
console.log(counter());
```

OUTPUT:

```
1
2
```

3. Array Methods & Higher-Order Functions

Essential Array Methods (map, filter, reduce)

```
const nums = [1, 2, 3, 4, 5];

// 1. map (transforms array)
const doubled = nums.map(x => x * 2);

// 2. filter (selects elements matching condition)
const evens = nums.filter(x => x % 2 === 0);

// 3. reduce (accumulates value)
const sum = nums.reduce((acc, curr) => acc + curr, 0);

console.log('Doubled:', doubled);
console.log('Evens:', evens);
console.log('Sum:', sum);
```

OUTPUT:

```
Doubled: [ 2, 4, 6, 8, 10 ]
Evens: [ 2, 4 ]
Sum: 15
```

4. Asynchronous JavaScript & Promises

Promises

```
const fetchData = () => {
  return new Promise((resolve, reject) => {
    let success = true;
    if (success) resolve("Data fetched!");
    else reject("Error occurred");
  });
};

fetchData()
  .then(res => console.log(res))
  .catch(err => console.error(err));
```

OUTPUT:

```
Data fetched!
```

Async / Await

```
async function getData() {
  try {
    const result = await fetchData();
    console.log('Async:', result);
  } catch (err) {
    console.log(err);
  }
}

getData();
```

OUTPUT:

```
Async: Data fetched!
```

5. Classes & Object-Oriented Programming

ES6 Classes & Inheritance

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `Hello, I'm ${this.name}`;
  }
}

class Developer extends Person {
  constructor(name, lang) {
    super(name);
    this.lang = lang;
  }
  code() {
    return `${this.name} codes in ${this.lang}`;
  }
}

const dev = new Developer('Thakur', 'JavaScript');
console.log(dev.greet());
console.log(dev.code());
```

OUTPUT:

```
Hello, I'm Thakur
Thakur codes in JavaScript
```

6. DOM Manipulation & Event Loop

DOM Selection & Events

```
// Selecting Elements
const btn = document.querySelector('#myBtn');

// Adding Event Listener
btn.addEventListener('click', (event) => {
  console.log('Button Clicked!', event.target);
});

// Modifying Content & Styles
btn.textContent = 'Click Me!';
btn.style.backgroundColor = '#38bdf8';
```

Event Loop Execution Order

```
console.log('1: Start');

setTimeout(() => console.log('2: Timeout (MacroTask)'), 1000);

Promise.resolve().then(() => console.log('3: Promise (MicroTask)'));

console.log('4: End');
```

OUTPUT:

```
1: Start
4: End
3: Promise (MicroTask)
2: Timeout (MacroTask)
```